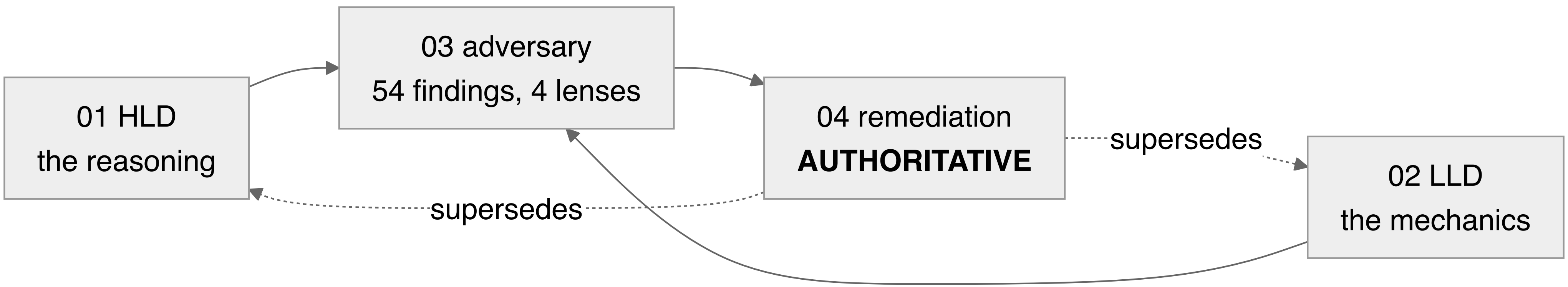
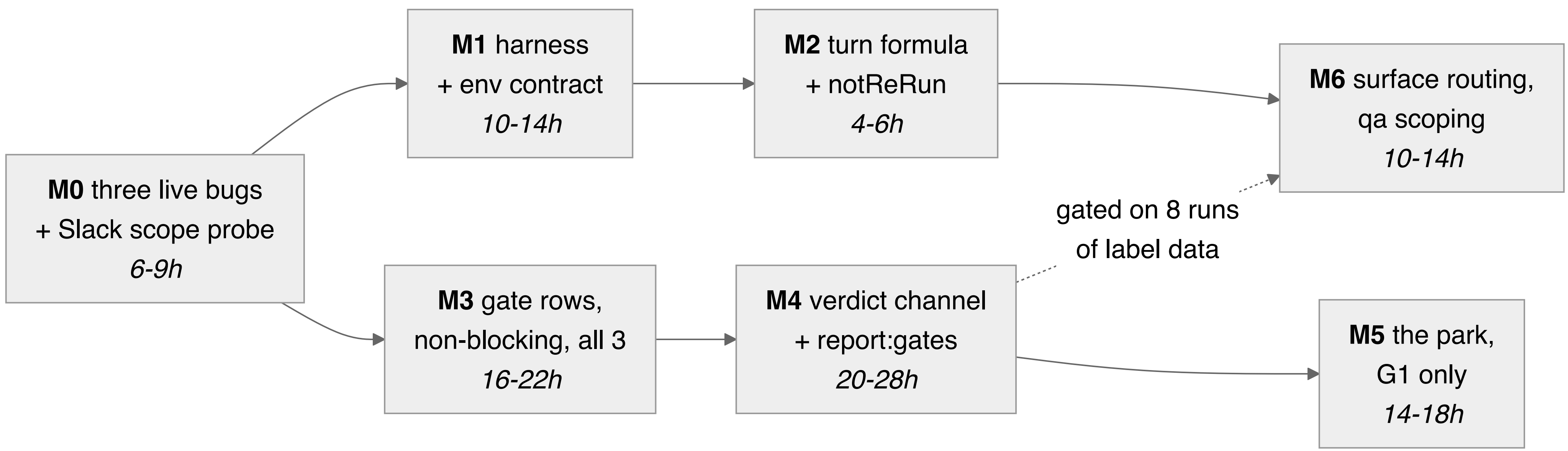
Diagram 0 — how the four documents relate.



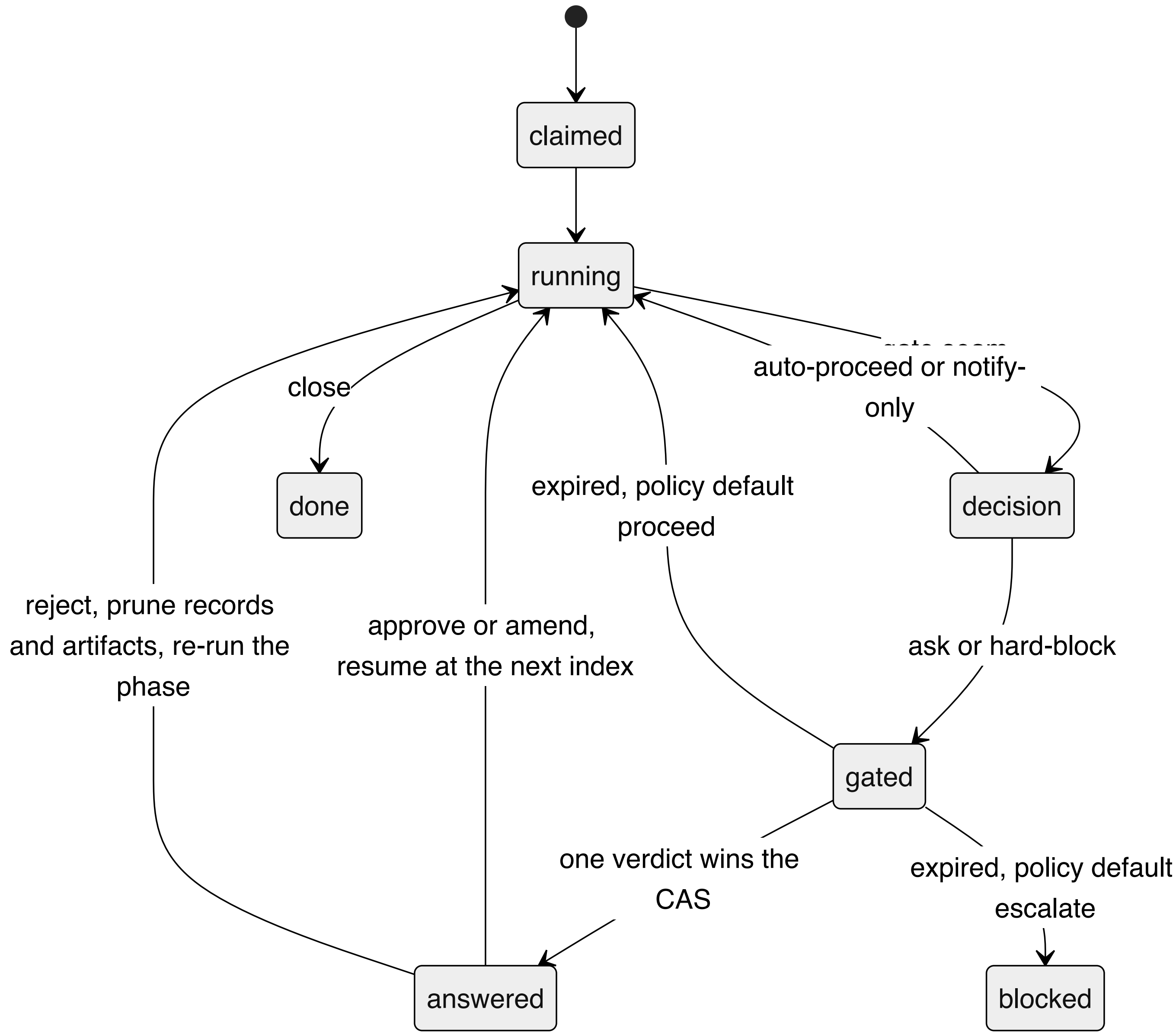
01 and 02 are the design as proposed; 03 is what four red teams did to it; 04 is the ruling and wins wherever it disagrees.

Diagram 4 — build order and what depends on what.



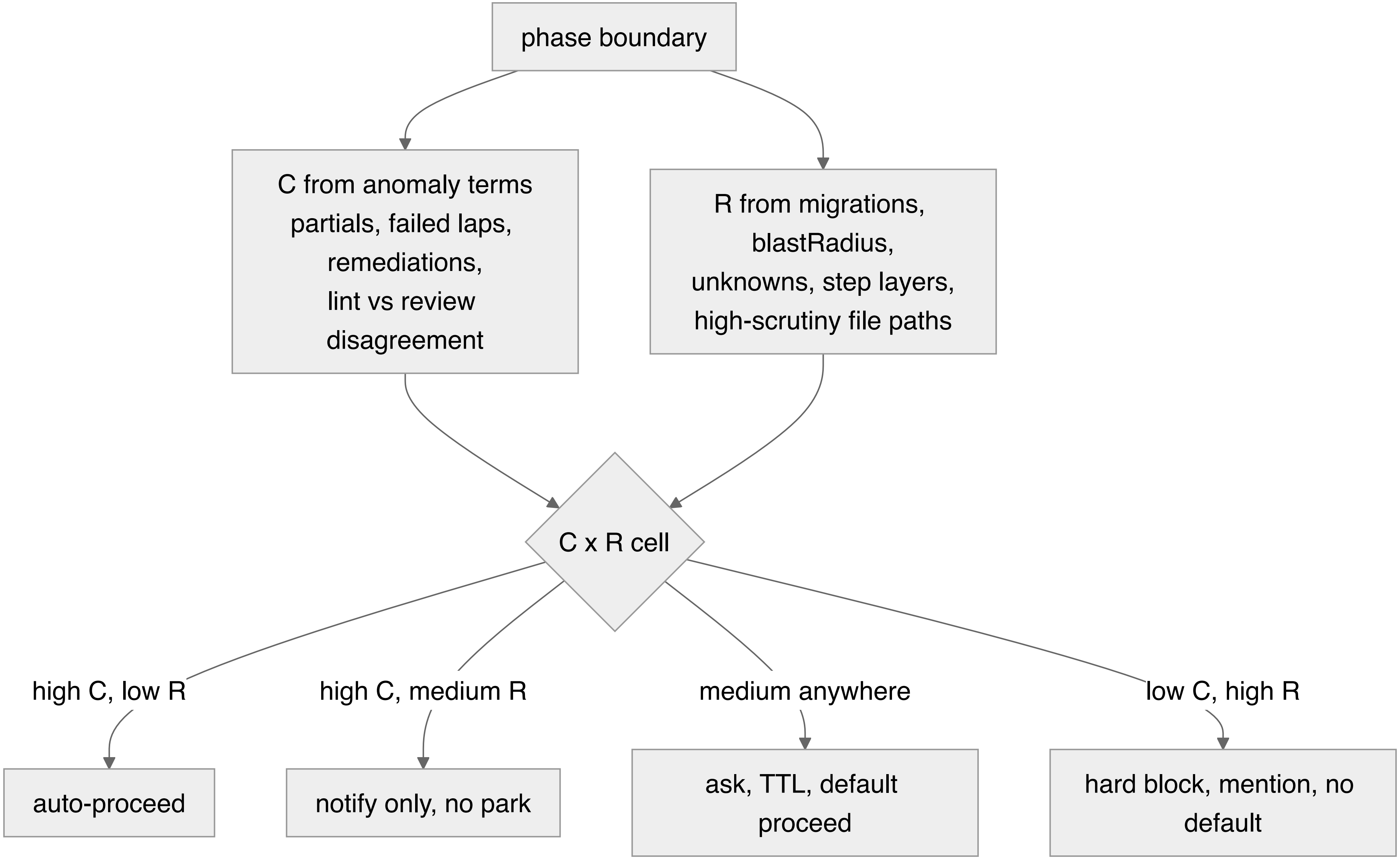
M0 is three bugs live in the shipped system today; two of them corrupt any gate data collected before they land, which is why nothing else may go first. M1–M2 are the UI-verification programme and need no human anywhere.

Diagram A — the run lifecycle as originally designed.



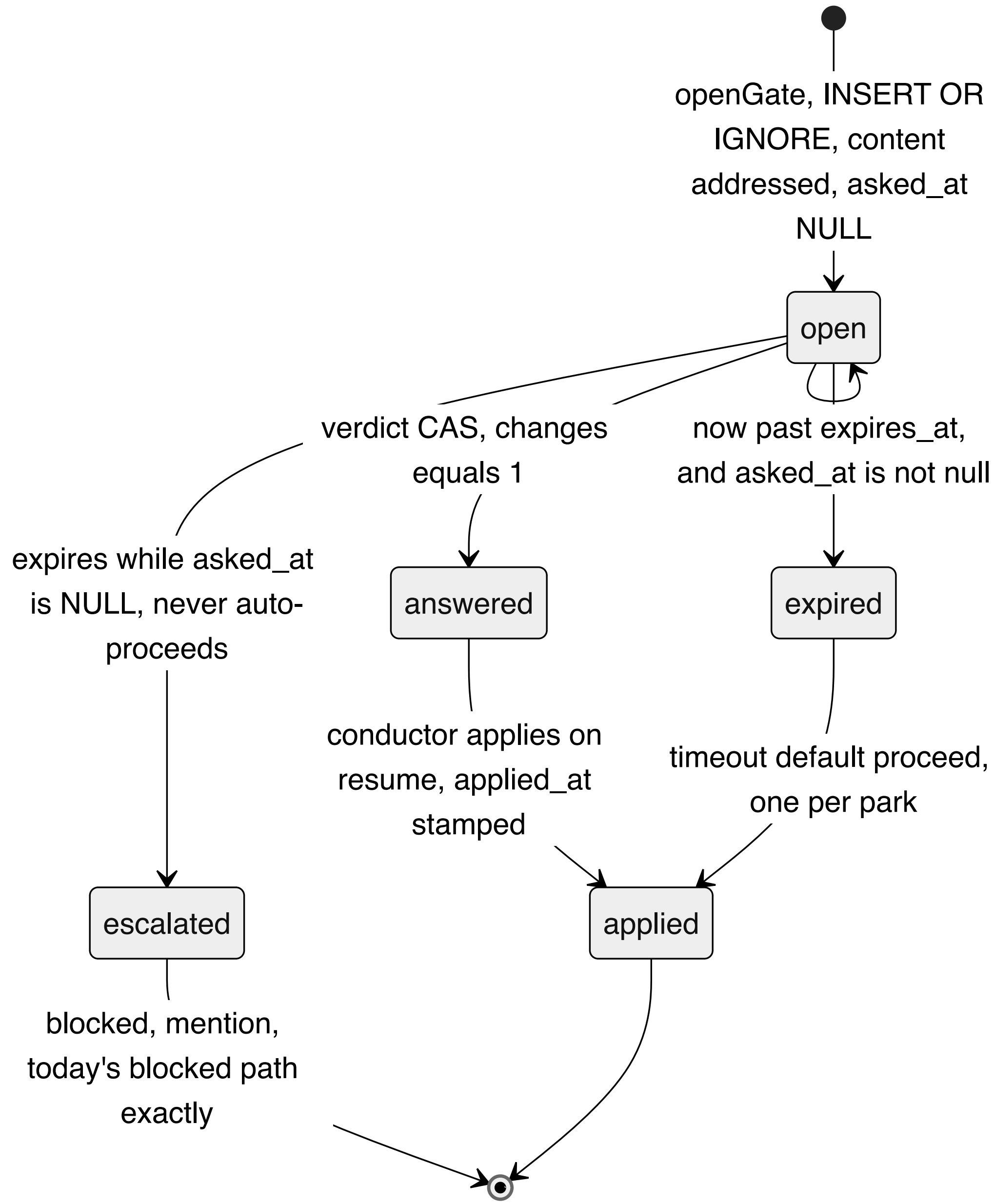
The same machine as the ASCII art above, drawn properly. Superseded by 04 §2: the aborted re-ask edge is deleted as false (resume enters at i=0 and shouldSkip walks past every ok record), the park moves behind a loop invariant, and G2/G3 stop parking at all. See Diagram 1 in 04 for what is actually being built.

Diagram B — how a boundary was to be classified.



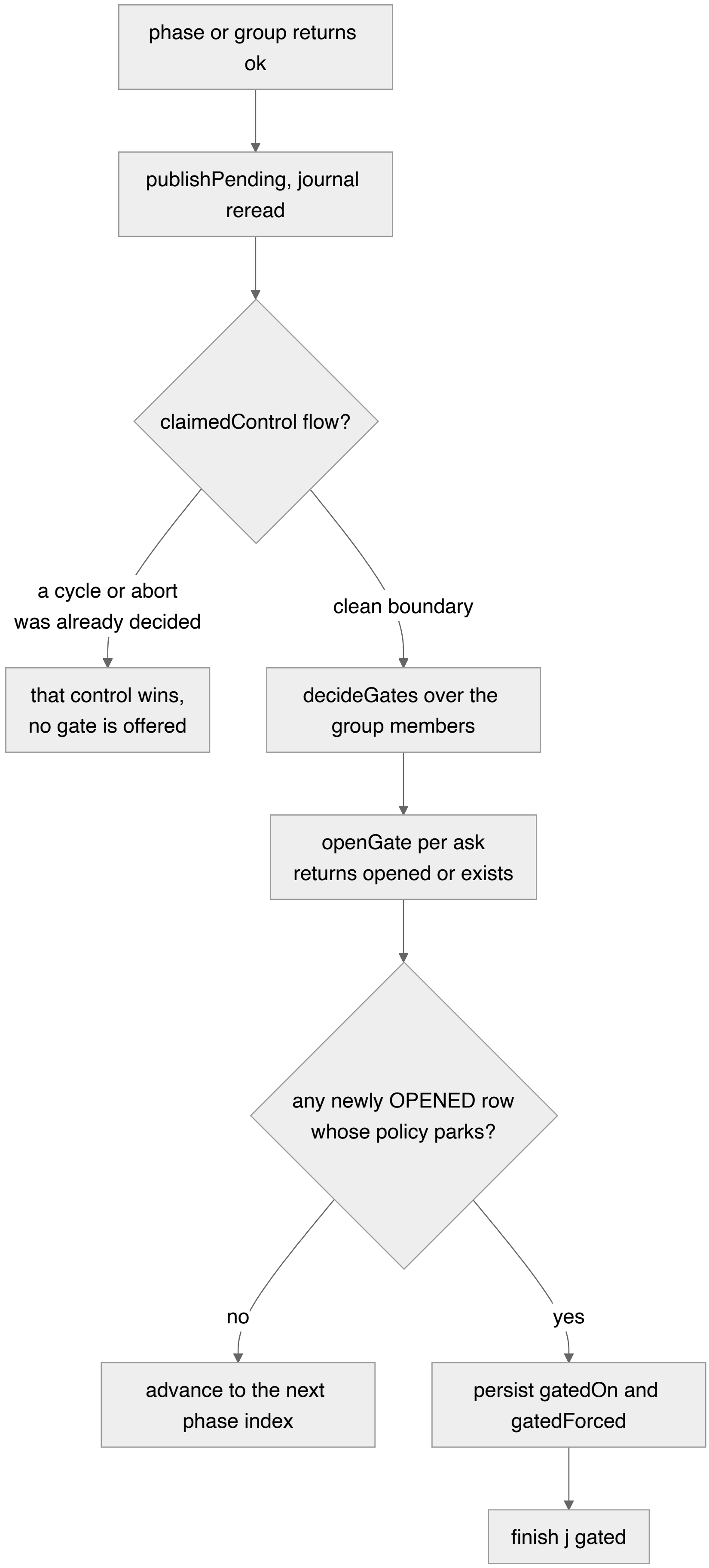
Deleted by 04 §2. Replayed against the four real runs, C reads high on 12 of 12 gate opportunities and R saturates at its floor on 4 of 4, so exactly one cell is reachable. Replaced by a single declared askProbability plus one non-random riskFloor override. C and R survive only as predictions that get scored.

Diagram C — the gate row lifecycle.



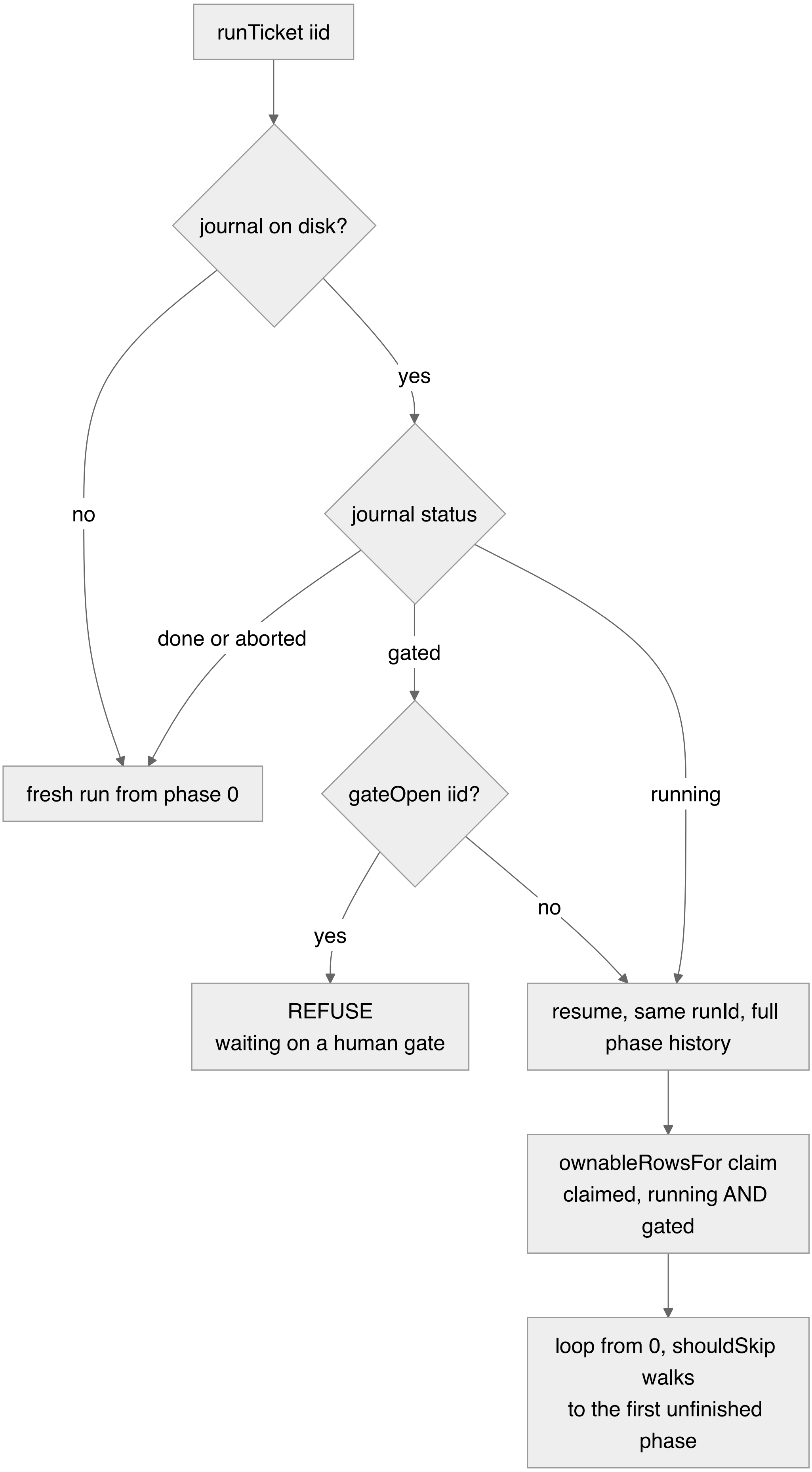
Every transition out of open is a compare-and-swap; only a statement that reports changes === 1 is allowed to act.

Diagram D — the seam, and why a gate is never a failure.



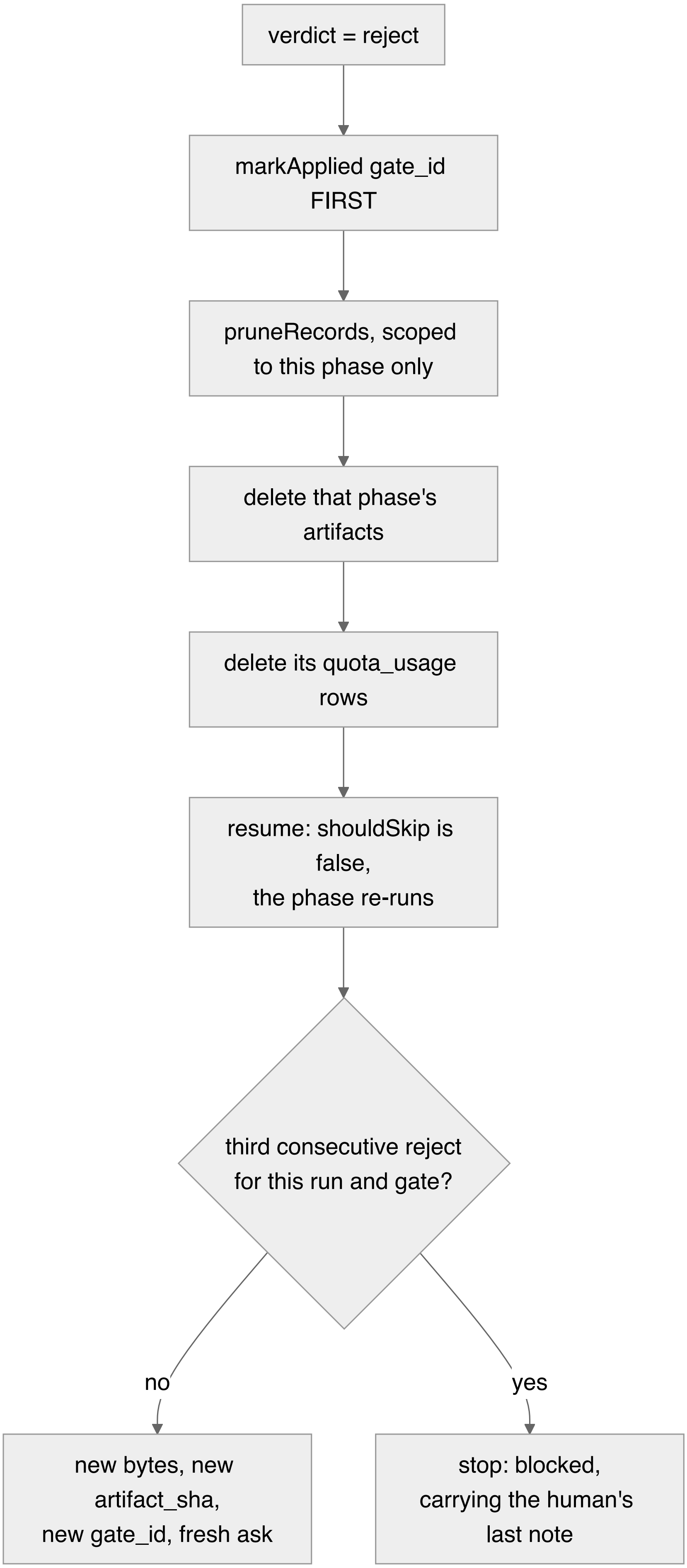
It sits after the group's single join and single reconciliation, so testcases and review are decided together and produce one park with two questions. It synthesises its own Control and never reaches afterFailure(), which is what makes a gate unable to spend a lap.

Diagram E — decideResume, including the --ticket hole.



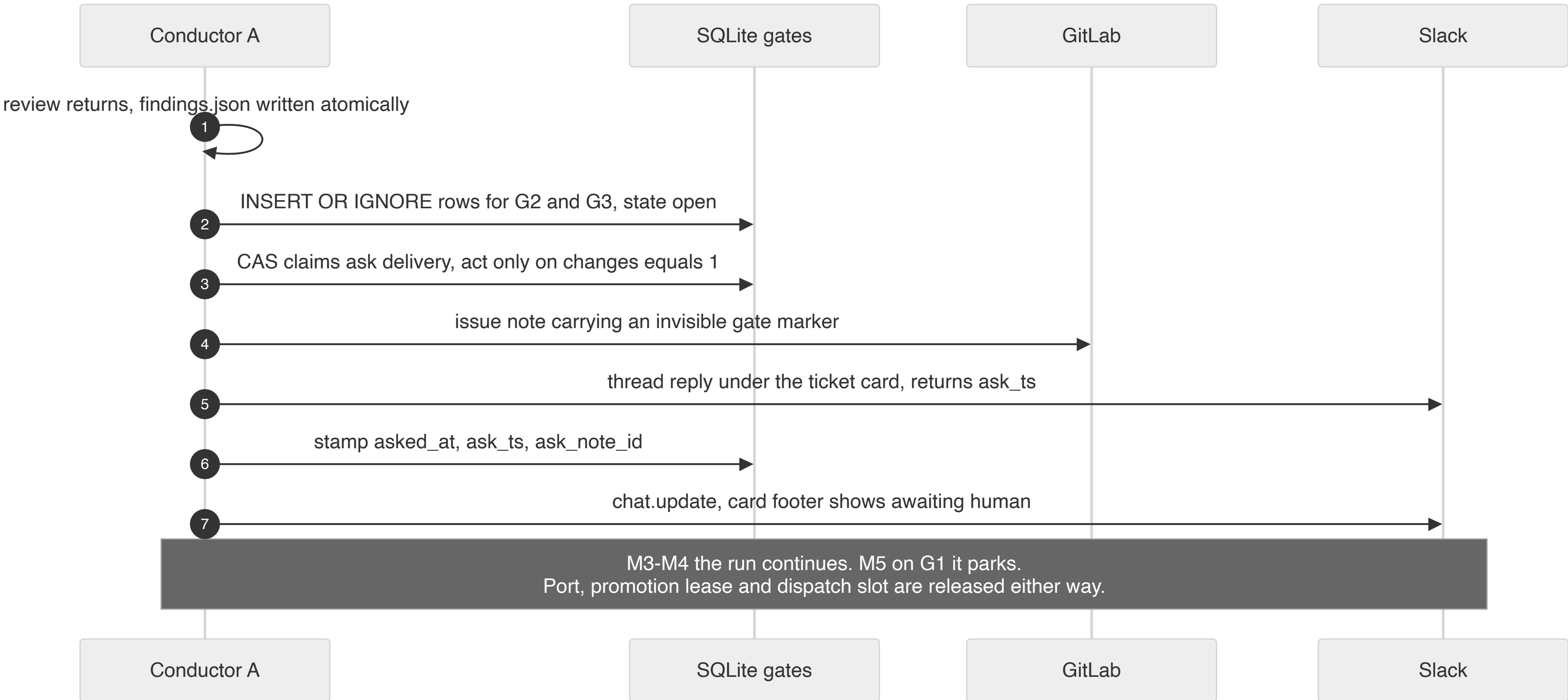
--ticket dispatches straight to runTicket and never passes the watcher, so the gate test is repeated here rather than trusted from isClaimed.

Diagram F — the reject path.



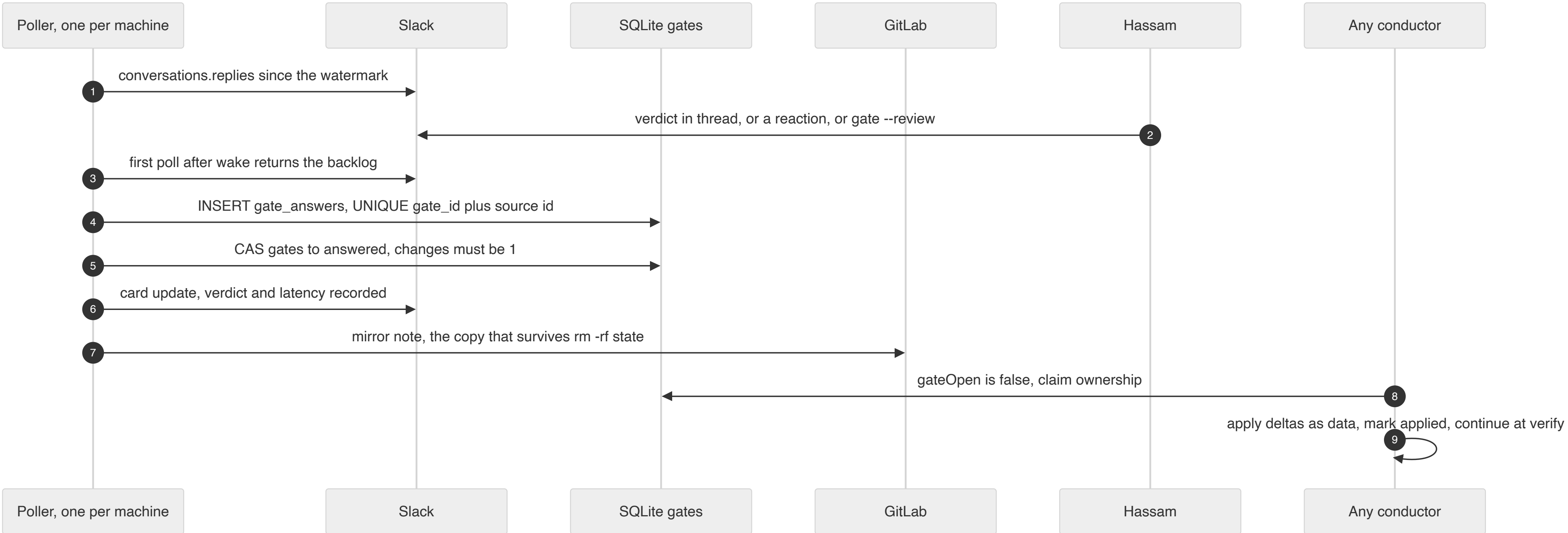
markApplied runs *before* the prune, so a crash between them re-prunes nothing the human already approved.

Diagram G·1 — one intervention, the ask.



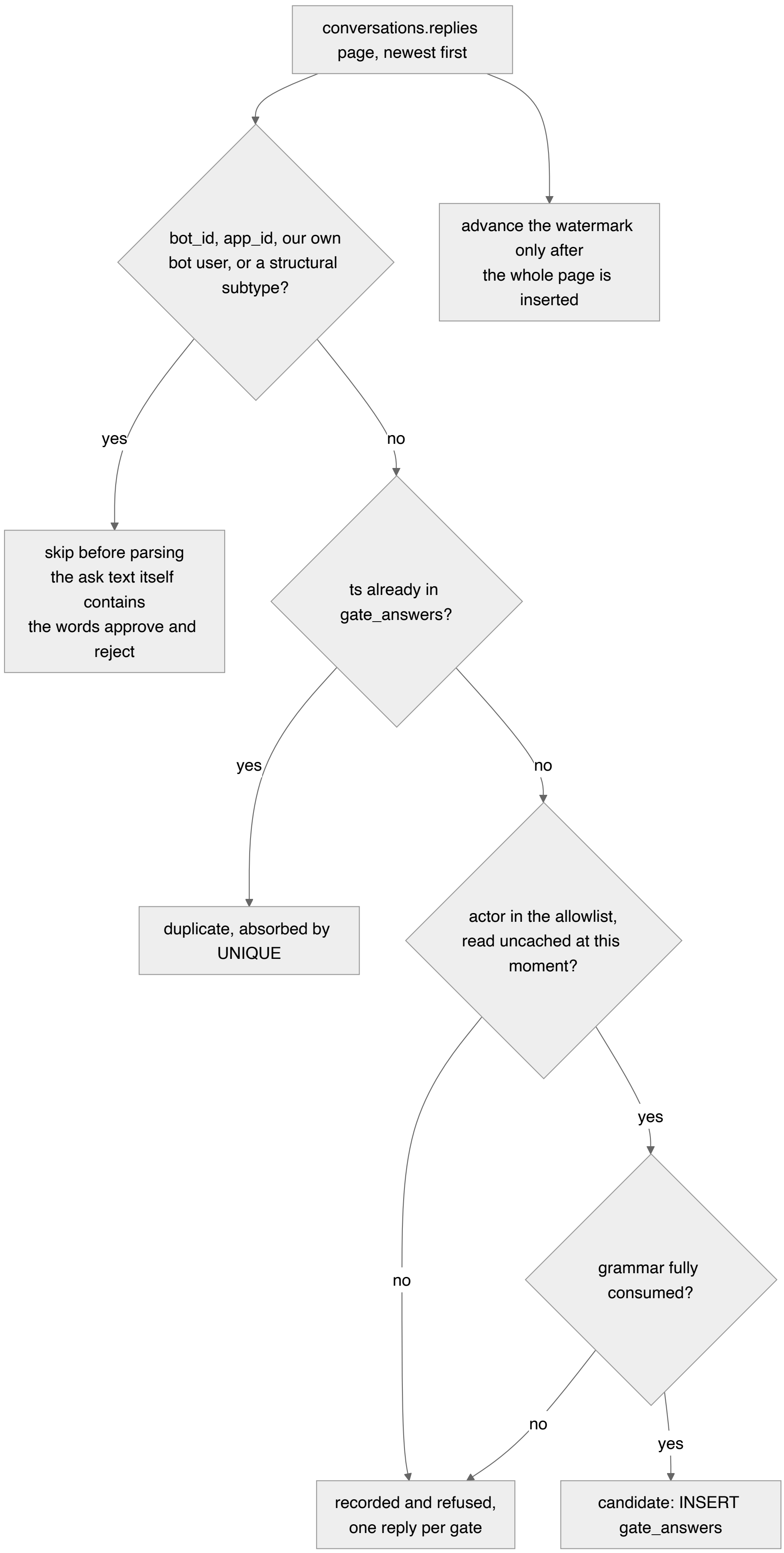
From the review artifact landing to the run parking (M5) or continuing (M3–M4). Everything here happens inside one conductor's phase boundary.

Diagram G·2 — one intervention, the verdict and the resume.



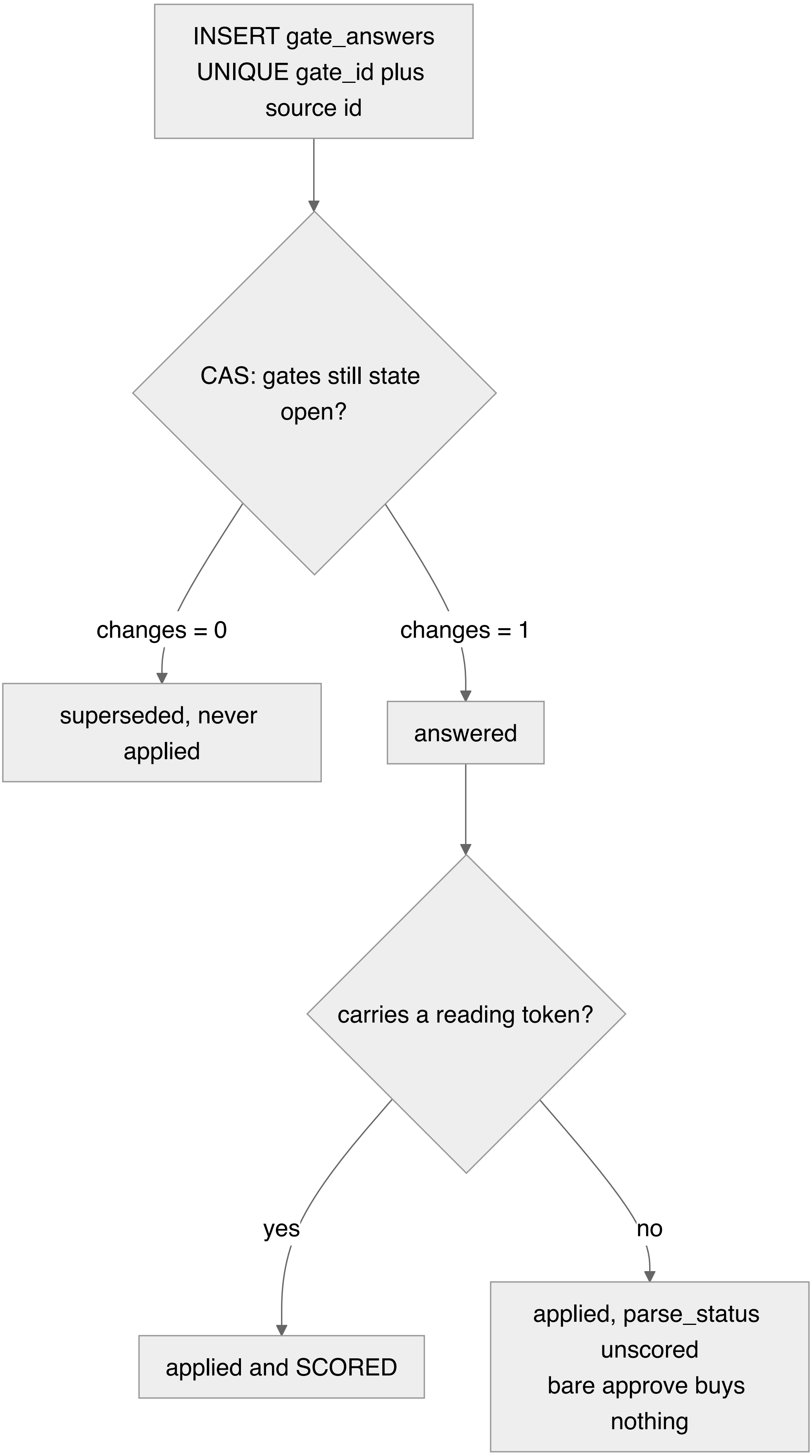
Between the ask above and the poll below, nothing is held and no process needs to be alive — the conductor that asked may be dead.

Diagram H·1 — verdict ingestion, the filters.



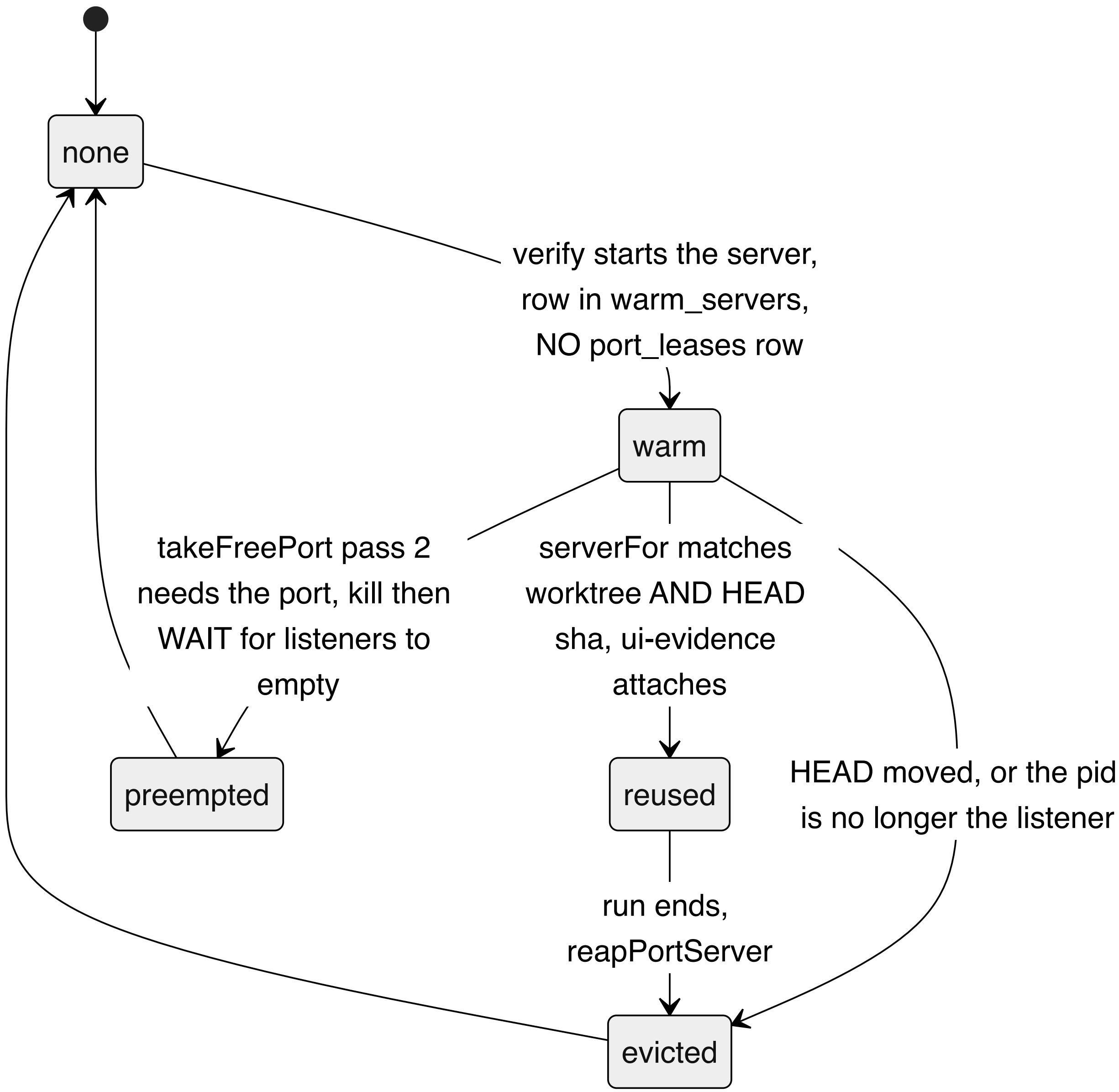
At-least-once delivery: a page may be read twice, by two conductors, across a sleep. Everything that is not a candidate is dropped before parsing.

Diagram H·2 — verdict ingestion, the application.



At-most-once application: the INSERT absorbs redelivery, the CAS absorbs everything else, and only a verdict carrying a reading token is scored.

Diagram I — the warm server is a cache, never a lease.



The invariant is that a warm server occupies no row in port_leases, so every existing reclaim path is untouched and a warm port can always be taken back by a run that actually needs it.

Diagram J — case routing, and where the same list stops being driven three times.

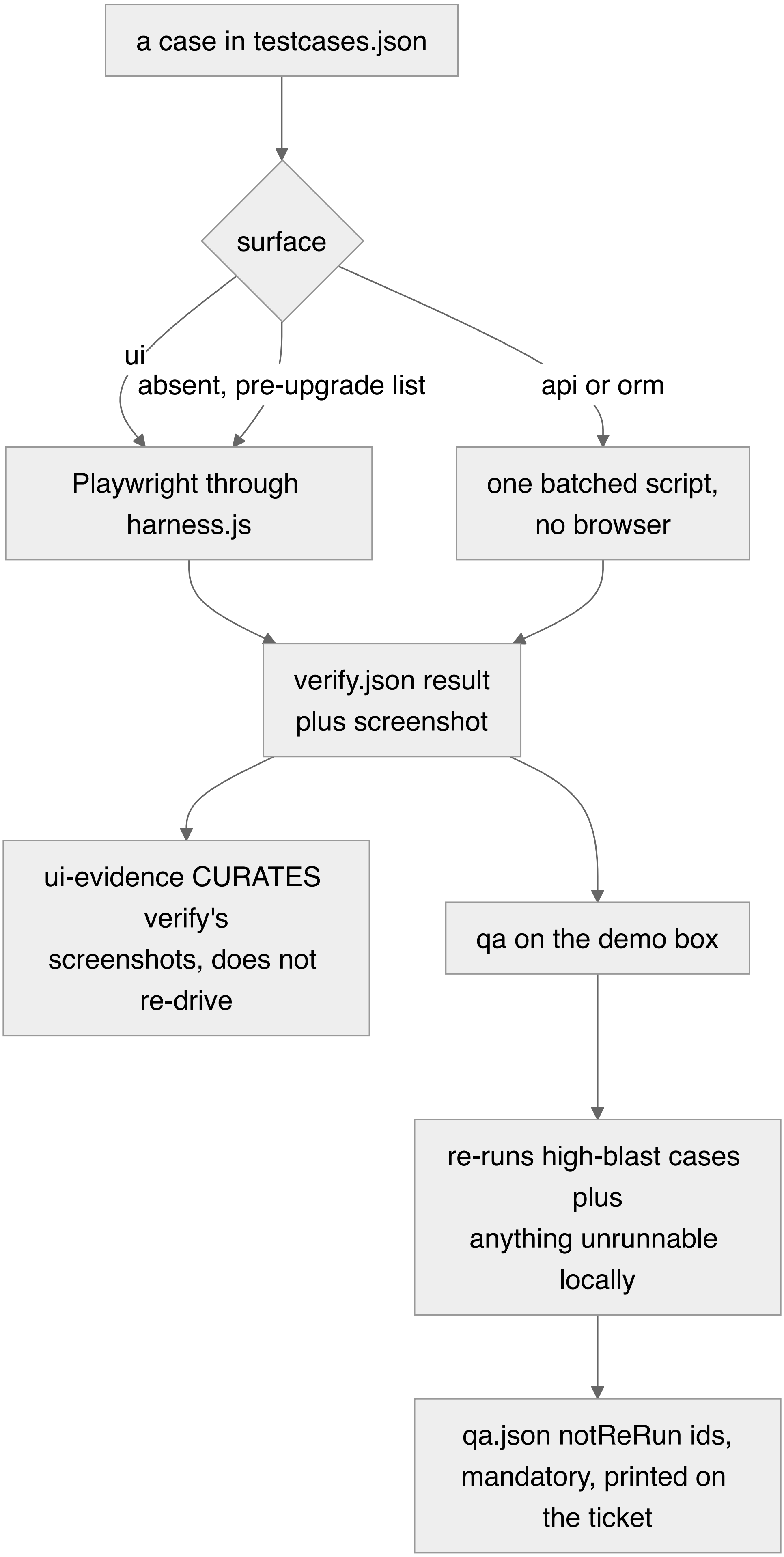
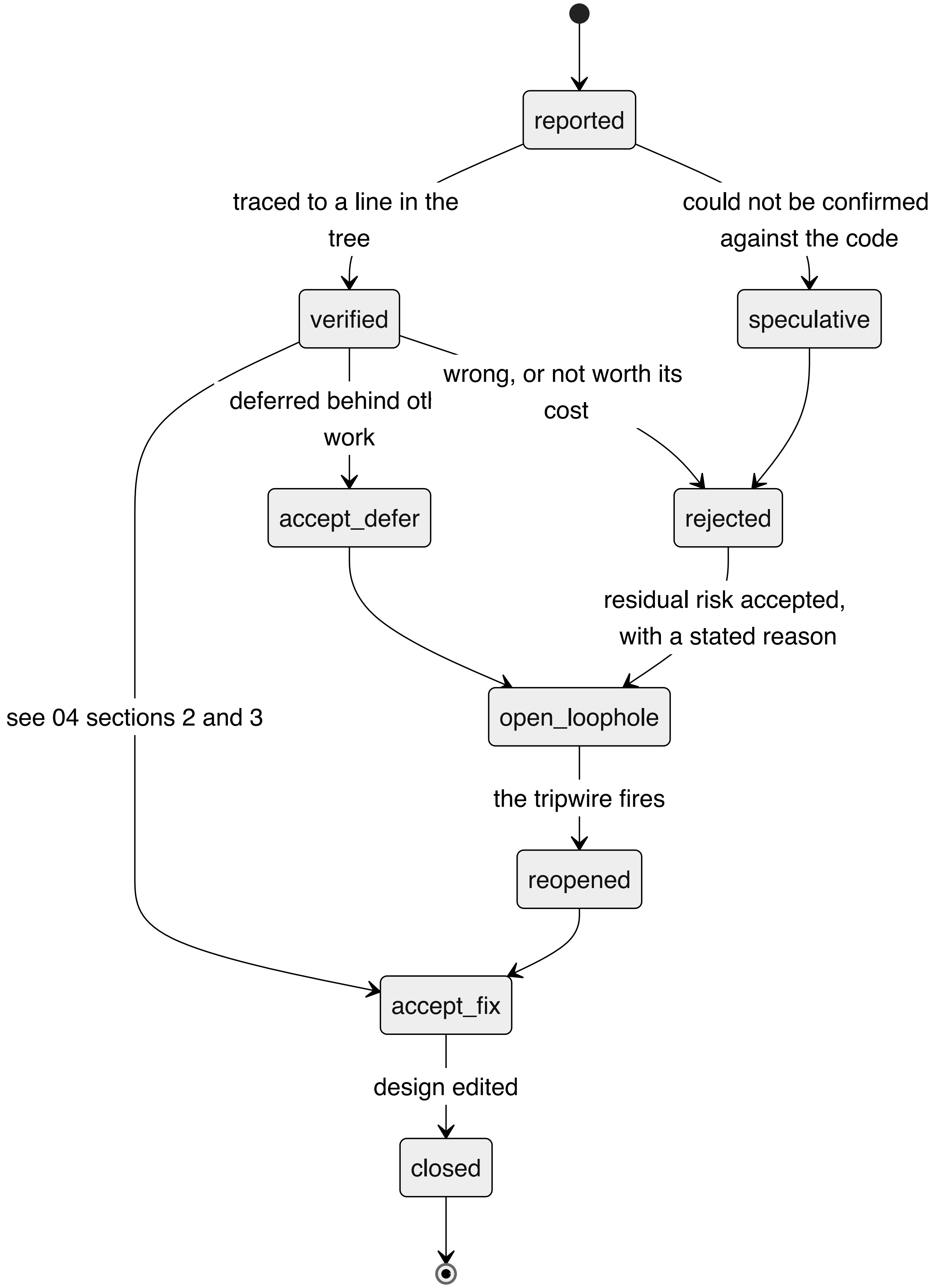
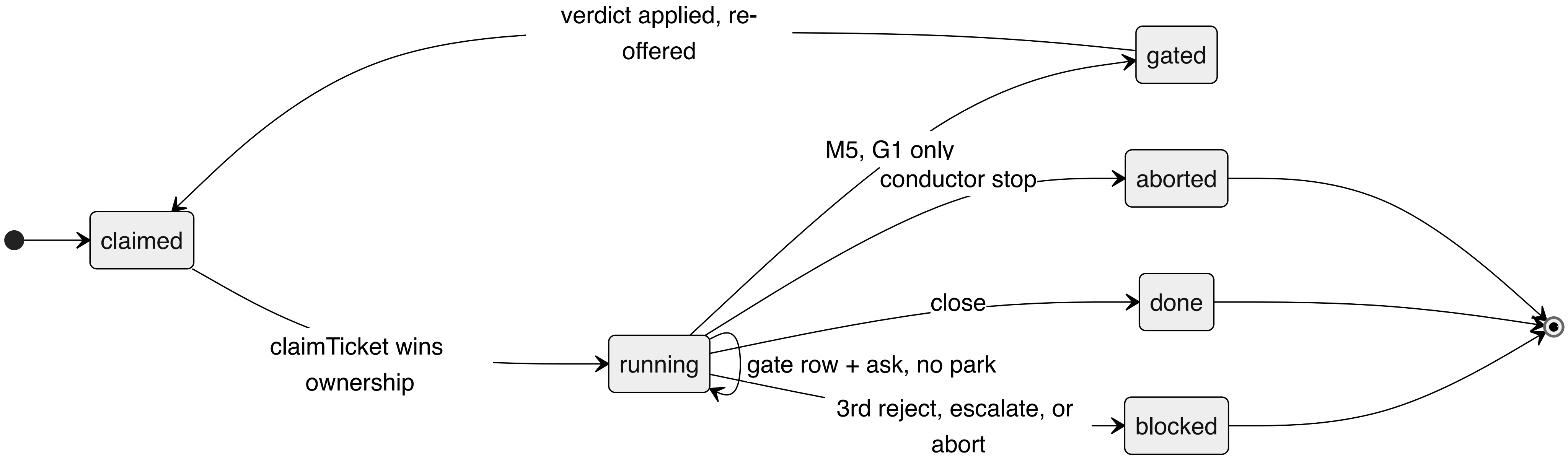


Diagram K — what happened to each of the 54 findings.



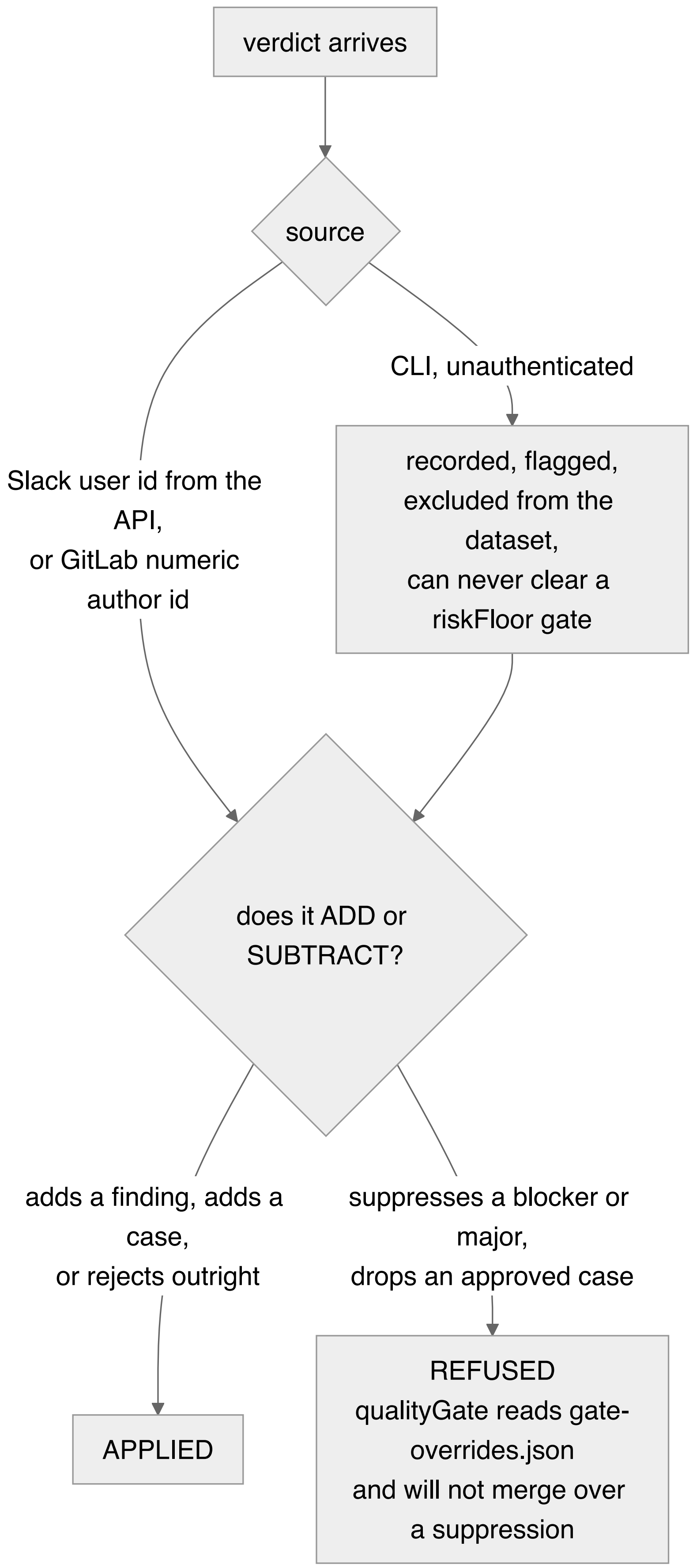
Nothing was dropped: a finding that is not fixed is carried, by name, into the open-loophole register with the tripwire that would reopen it.

Diagram 1 — the run lifecycle as ruled.



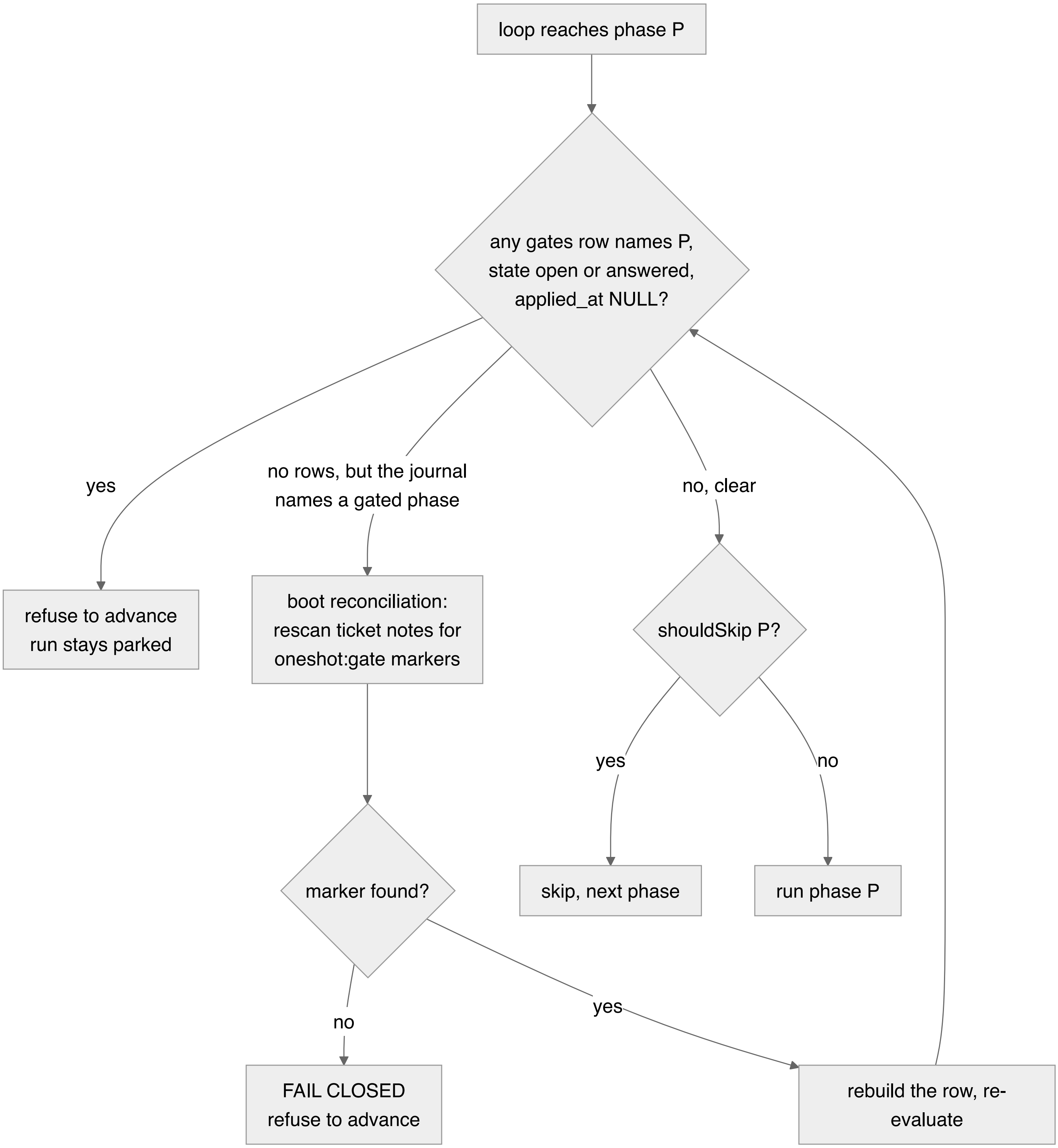
The important edge is the self-loop: in M3–M4 a boundary writes a gate row and posts a real ask, and the run *keeps going*. Only M5 introduces a park, only on G1, and only when askProbability selects it or riskFloor forces it.

Diagram 3 — the authority rule: a verdict may add, never subtract.



A gated phase runs bypassPermissions with Bash unguarded by write-scope, so it can forge its own approval. The fix that actually works is removing the incentive rather than the capability: the worst a forged verdict can do is make the pipeline work harder.

Diagram 2 — the park is a loop invariant, not a status.



This is the D-02 / S-11 fix. A status flag on a non-atomically written journal desynchronises on a hard SIGINT, a boot reap, or the rm -rf state/ that db.ts:2 blesses — and then shouldSkip walks over three green records to merge. The check therefore lives inside the loop that acts, immediately before shouldSkip at runner.ts:718.